

Petclinic on Rancher Desktop (Win11): Local Kubernetes + PostgreSQL (Helm) Runbook

End-to-end notes from a working local setup: Bitnami PostgreSQL deployed in-cluster (with a PVC) and the Petclinic app deployed in the same namespace, pulling its image from the GitLab registry and sourcing DB credentials from a Kubernetes Secret.

This guide is intentionally practical: steps, commands, the blockers you hit, and the fixes that were applied.

What you end up with

- Namespace: **petclinic**
- PostgreSQL: Bitnami chart (StatefulSet + Service + PVC) reachable as **petclinic-db-postgresql:5432** inside the cluster
- DB credentials: Secret **petclinic-db-credentials** (keys: **POSTGRES_USER**, **POSTGRES_PASSWORD**, **POSTGRES_DB**)
- App: Deployment **petclinic** pulling from **gitlab-registry.endava.com** using image pull Secret **gitlab-registry**
- App config split: ConfigMap for non-sensitive settings; Secret refs for DB username/password

Prerequisites

- Rancher Desktop with Kubernetes enabled (k3s).
- kubectl configured to point to the Rancher Desktop cluster context.
- Helm installed locally.
- Access to the GitLab container registry for the petclinic image (PAT/token kept locally, not in Git).

Rancher Desktop runs Kubernetes on containerd. If you ever work with a local registry, you typically use nerdctl and containerd registry config. For this runbook, the image is pulled from GitLab.

1) Namespace basics (and the namespace typo pitfall)

Kubernetes objects are namespaced. Secrets are namespaced too: a Deployment in **petclinic** cannot reference a Secret in another namespace (e.g., **registry**).

```
kubectl get ns petclinic
kubectl create ns petclinic
```

If kubectl says 'No resources found' and you expected resources, double-check the namespace spelling and context (you had a **petclinig** typo at one point):

```
kubectl config current-context
kubectl get ns
kubectl -n petclinic get deploy,pod,svc
```

2) GitLab image pull secret (and how you verified it)

Your pod events showed: **FailedToRetrieveImagePullSecret / ImagePullBackOff**. The working fix was to create the docker-registry secret in the **petclinic** namespace and reference it from the Deployment.

```
kubectl -n petclinic create secret docker-registry gitlab-registry `
  --docker-server=gitlab-registry.endava.com `
  --docker-username="<your.email@endava.com>" `
  --docker-password="<your_PAT_or_token>" `
  --docker-email="<your.email@endava.com>"

kubectl -n petclinic get secret gitlab-registry
```

How you checked the pod events:

```
kubectl -n petclinic describe pod -l app=petclinic
```

After fixing the secret, you forced a retry by restarting the Deployment (or deleting the pods):

```
kubectl -n petclinic rollout restart deploy/petclinic
kubectl -n petclinic delete pod -l app=petclinic
```

3) PostgreSQL via Helm (Bitnami)

Repo setup

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update
```

Helm release behaviour you saw

- Release name is Helm's identity for an install in a namespace (you used: **petclinic-db**).
- Trying to install again with the same release name gives 'cannot reuse a name that is still in use'.
- Use **upgrade** to re-apply changes, or **uninstall** if you want to remove chart-managed resources.

```
helm -n petclinic list
helm -n petclinic upgrade petclinic-db bitnami/postgresql -f .\postgres-values.yaml
helm -n petclinic uninstall petclinic-db
```

Install (the working command)

```
helm -n petclinic install petclinic-db bitnami/postgresql -f .\postgres-values.yaml
```

Why your auth settings mattered

You used an existing Secret (**petclinic-db-credentials**) and explicitly set the username and database in the Helm values. That combination is what led to container log lines like:

- Changing password of postgres
- Creating user petclinic
- Granting access to database petclinic

You confirmed creation inside Postgres with:

```
kubectl -n petclinic logs petclinic-db-postgresql-0 --tail=200
kubectl -n petclinic exec -it petclinic-db-postgresql-0 -- bash -lc "psql -U postgres -d postgres -c '\du'"
kubectl -n petclinic exec -it petclinic-db-postgresql-0 -- bash -lc "psql -U postgres -d postgres -c '\l'"
```

Quick resource checks

```
kubectl -n petclinic get pods
kubectl -n petclinic get svc
kubectl -n petclinic get pvc
```

4) Core concepts behind the DB deployment

StatefulSet and ordinals

Postgres is deployed as a StatefulSet so each replica has a stable identity and stable storage.

- Pod names are stable: **petclinic-db-postgresql-0**, -1, etc.
- Restarting the pod preserves the ordinal (-0 stays -0).
- Scaling up creates new ordinals (-1, -2, etc.).

PVCs and persistence

With **primary.persistence.enabled=true**, Kubernetes creates a per-pod PVC from a template. The PVC binds to a PV (local-path storage class in Rancher Desktop), and that PV holds the database files.

5) Secrets: reading, decoding, and tracking where they came from

Decoding and verifying Secret values (PowerShell)

```
kubectl -n petclinic get secret petclinic-db-credentials -o jsonpath="{.data}"

$pwB64 = kubectl -n petclinic get secret petclinic-db-credentials `
  -o "jsonpath={.data.POSTGRES_PASSWORD}"
$pw = [Text.Encoding]::UTF8.GetString([Convert]::FromBase64String($pwB64))
```

Using the decoded password without interactive prompts:

```
kubectl -n petclinic exec -it petclinic-db-postgresql-0 -- `
  bash -lc "PGPASSWORD='$pw' psql -U petclinic -d petclinic -h 127.0.0.1 -p 5432 -c 'select
  1;'"
```

Tracing a Secret like postgresql-auth (your 'leftover?' question)

A quick way to see whether a Secret was created by a controller/chart is to check labels and ownerReferences.

```
kubectl -n petclinic get secret postgresql-auth -o jsonpath="{.metadata.labels}"
kubectl -n petclinic get secret postgresql-auth -o jsonpath="{.metadata.ownerReferences}"
kubectl -n petclinic get secret postgresql-auth -o yaml
```

If a Secret has no ownerReferences and no Helm/chart labels, it's usually something that was created manually or by a different process earlier.

6) Persistence proof (repeatable check)

You validated persistence by creating a tiny table + row, restarting the DB pod, then confirming the table still exists.

```
$pwB64 = kubectl -n petclinic get secret petclinic-db-credentials `
  -o "jsonpath={.data.POSTGRES_PASSWORD}"
$pw = [Text.Encoding]::UTF8.GetString([Convert]::FromBase64String($pwB64))

kubectl -n petclinic exec -it petclinic-db-postgresql-0 -- `
  bash -lc "PGPASSWORD='$pw' psql -U petclinic -d petclinic -c 'create table if not exists
  persist_check(id int primary key);'"

kubectl -n petclinic exec -it petclinic-db-postgresql-0 -- `
  bash -lc "PGPASSWORD='$pw' psql -U petclinic -d petclinic -c 'insert into
  persist_check(id) values (1) on conflict do nothing;'"

kubectl -n petclinic delete pod petclinic-db-postgresql-0
kubectl -n petclinic get pods -w

kubectl -n petclinic exec -it petclinic-db-postgresql-0 -- `
  bash -lc "PGPASSWORD='$pw' psql -U petclinic -d petclinic -c 'select * from
  persist_check;'"
```

7) Petclinic app configuration: environment variables explained

These variables were present in your setup and are worth understanding:

- **SPRING_PROFILES_ACTIVE=postgres** - activates the Spring profile named postgres, which commonly switches datasource settings and other beans.
- **SPRING_DATASOURCE_URL** - JDBC endpoint the app uses. In-cluster it points to the Postgres Service DNS: **jdbc:postgresql://petclinic-db-postgresql:5432/petclinic**.
- **FLYWAY_ENABLED=true** - controls **spring.flyway.enabled** via an application.yaml placeholder (**\${FLYWAY_ENABLED:false}**). Flyway migrations run only if Flyway is on the classpath and migrations exist.
- **SQL_INIT_MODE=never** and **SPRING_SQL_INIT_MODE=never** - disables Spring's SQL init (schema.sql/data.sql) so it doesn't clash with Flyway.
- **HIBERNATE_DDL_AUTO=validate** - Hibernate verifies the schema matches the entities; it does not create or update tables.

Why this mattered for 'no tables'

With **HIBERNATE_DDL_AUTO=validate**, the app will not create tables on its own. Tables appear only if Flyway runs migrations (or if you create them manually).

Caution: the Flyway env var name can silently disable migrations

In this setup, **application.yaml** uses **spring.flyway.enabled: \${FLYWAY_ENABLED:false}**. That means Spring looks for an environment variable literally named **FLYWAY_ENABLED**.

If you set **SPRING_FLYWAY_ENABLED=true** in a ConfigMap, it will be ignored by that placeholder and Flyway will default to **false** (no migrations, often no obvious logs).

Fix: set **FLYWAY_ENABLED: "true"** in the ConfigMap (or remove the placeholder and rely on Spring's relaxed binding).

Optional: make Flyway logs visible

If migrations are running but you want explicit output, add this to **application.yaml**:

```
logging:
  level:
    org.flywaydb: INFO
```

8) App uses Secret-based DB credentials (and how you verified it)

You confirmed in **kubectl describe pod** that the container env vars were sourced from the Secret **petclinic-db-credentials** (keyRefs), not hardcoded.

```
kubectl -n petclinic describe pod -l app=petclinic
```

In the describe output you saw mappings like:

```
SPRING_DATASOURCE_USERNAME: <set to key 'POSTGRES_USER' in secret  
'petclinic-db-credentials'>  
SPRING_DATASOURCE_PASSWORD: <set to key 'POSTGRES_PASSWORD' in secret  
'petclinic-db-credentials'>
```

9) Applying / reapplying manifests (without pasting YAML into this PDF)

Your working loop was:

- Apply manifests with `kubectl apply` (deployment/service/configmap, etc.).
- Create/update Secrets from the CLI (or manage them separately) so you don't store credentials in Git.
- Restart the Deployment to pick up new env/config.

```
kubectl -n petclinic apply -f .  
kubectl -n petclinic rollout restart deploy/petclinic  
kubectl -n petclinic rollout status deploy/petclinic --timeout=180s
```

If you want to remove and recreate only the app resources (keeping the DB and PVC):

```
kubectl -n petclinic delete deploy/petclinic svc/petclinic `\  
  configmap/petclinic-config --ignore-not-found  
kubectl -n petclinic apply -f .
```

10) Troubleshooting commands you used the most

See why a Pod isn't ready / why it restarts:

```
kubectl -n petclinic describe pod -l app=petclinic  
kubectl -n petclinic get events --sort-by=.metadata.creationTimestamp
```

Check logs (including crash loop previous container logs):

```
kubectl -n petclinic logs deploy/petclinic --tail=200  
kubectl -n petclinic logs -l app=petclinic --tail=200 --all-containers  
kubectl -n petclinic logs pod/<pod-name> --previous --tail=200
```

Interpret readiness probe failures

Your readiness probe hit **/actuator/health/readiness** and returned 'connection refused' when the app process exited during startup. Once the process stays up and binds to 8080, readiness becomes green and the Service begins routing traffic.

11) DB schema checks (Flyway ran vs DB still empty)

You checked for tables and found none:

```
kubectl -n petclinic exec -it petclinic-db-postgresql-0 -- `
  bash -lc "psql -U petclinic -d petclinic -c '\dt'"
```

A strong indicator that Flyway ran is the presence of **flyway_schema_history**:

```
kubectl -n petclinic exec -it petclinic-db-postgresql-0 -- `
  bash -lc "psql -U petclinic -d petclinic -c \"select
    to_regclass('public.flyway_schema_history');\""
```

You also verified the app was configured with the postgres profile, the correct JDBC URL, and Flyway enabled:

```
kubectl -n petclinic exec -it deploy/petclinic -- sh -lc `
  "printenv | grep -E 'FLYWAY_ENABLED|SPRING_PROFILES_ACTIVE|SPRING_DATASOURCE_URL' || true"
```

Your container image didn't have the jar CLI (jar: not found). That usually means the image includes a JRE/runtime but not the full JDK tools. A practical check (if unzip exists) is:

```
kubectl -n petclinic exec -it deploy/petclinic -- sh -lc `
  "command -v unzip >/dev/null && unzip -l /app/app.jar `
  | grep -E 'db/migration|flyway' | head -n 50 || echo 'unzip not available'"
```

12) Local access

Expose the Service to localhost:

```
kubectl -n petclinic port-forward svc/petclinic 8080:80
# Browser: http://localhost:8080
```

If the UI shows an internal error for a specific page/menu action, reproduce it and immediately check the last few hundred lines of app logs for the corresponding stack trace:

```
kubectl -n petclinic logs deploy/petclinic --tail=300
```

Quick checklist

- DB pod ready: **petclinic-db-postgresql-0** is Running 1/1.
- PVC bound: **data-petclinic-db-postgresql-0** is Bound (storageClass local-path).
- DB connection works using secret-sourced password: **select 1** returns 1 for user petclinic.
- App pod ready: **petclinic** is Running 1/1; logs show HikariPool started and JDBC URL points to petclinic-db-postgresql.
- Local access works via port-forward to **svc/petclinic**.

Appendix: Local registry notes (from your earlier registry guide)

These notes are included because Rancher Desktop uses containerd. When you deal with images locally, nerdctl matches the runtime; the Docker CLI talks to dockerd, which isn't used in this mode.

- If you deploy a registry inside the cluster, a common pattern is to expose it via NodePort to localhost (for example, <http://localhost:32000>).
- A registry stores image layers and tags; containerd keeps a local image cache; you move images between registries with pull/tag/push.
- ImagePullBackOff troubleshooting is the same theme: Kubernetes needs access to the registry (auth + DNS/port), and the image reference must match what exists in that registry.

Typical nerdctl flow when copying an image from GitLab to a local registry:

```
# Log in to GitLab registry (uses your PAT/token)
nerdctl login gitlab-registry.endava.com

# Pull remote image
nerdctl pull gitlab-registry.endava.com/<group>/<image>:latest

# Tag it for the local registry
nerdctl tag gitlab-registry.endava.com/<group>/<image>:latest `
    localhost:32000/<image>:latest

# Push into the local registry
nerdctl push localhost:32000/<image>:latest
```

If you expose the local registry through a Kubernetes Service, confirm it is reachable and that the tag exists before pointing workloads at it:

```
kubect1 -n registry get pods,svc
# from Windows:
curl http://localhost:32000/v2/_catalog
```